

UROP Project Report

UROP - Report

Visualisation of the Code Review Process

Daniel Bee

A report submitted in part fulfilment of the scholarship of

BSc (Hons) in Computer Science

Supervisor: DongGyun Han



Department of Computer Science
Royal Holloway, University of London

June 26, 2024

Declaration

This report has been prepared on the basis of my own work. Where other published and unpublished source materials have been used, these have been acknowledged.

Word Count: TBC

Student Name: Daniel Bee

Date of Submission: TBC

Signature: Daniel Bee

Table of Contents

Abstract 3

Project Specification 4

1 Introduction 5

 1.1 Project Prototype Screenshot 5

2 Crawler 6

3 Database 7

4 Frontend 8

5 Backend 9

6 Conclusion 10

Abstract

This report covers the design and development cycle of the "Visualising Interactions Between Developers During Code Review" UROP project. This includes the thought processes throughout the project, as well as the many challenges that were faced along the way and how these were dealt with. There are four primary milestones that were allocated and scheduled for this project: Crawler, Database, Frontend, and Backend. Firstly, the Crawler milestone was where pull request information from the TypeScript GitHub repository from Microsoft was fetched via the GitHub GraphQL API. Secondly, the Database milestone was the process of designing an appropriate relational database schema to hold this newly retrieved pull request information for easier analysis. Thirdly, the Frontend milestone was where mock data was inputted into a forced-directed graph powered by D3.js and React to replace this mock data with the pull request information from the relational database later on. Fourthly, and finally, the Backend milestone was the section of the project where everything came together. During this milestone, the frontend graph was linked to the backend API endpoint to allow data to be transferred to a callback from axios, a promise-based HTTP client for browsers and node.js.

Project Specification

The UROP project aims to create a full stack, web-based system that allows users to view the relationships between developers during the Code Review process. Python is used to code the initial crawling stage of the project, making use of the requests pip library to make calls to the GitHub GraphQL API endpoint with an appropriate template string. Python is also used in the migration stage of the project to move the newly acquired pull request information from individual JSON files to an efficient relational database, using the mysql-connector-python pip library to perform this. The frontend handles the visualising of the data itself, and is primarily coded in TypeScript, whilst the backend, coded in Java, makes queries to a local relational database to consummate the visualisation. Axios comes in handy for the frontend stage of the project to make the call to the backend API endpoint, and allows users to easily add parameters to a callback. D3.js is an invaluable tool in deciphering the response Axios receives from that callback, and subsequently produces the final visualisation the user sees. Lastly, Spring Boot was vital in allowing easy development of the REST API building blocks that were essential to the development of the project. This is particularly clear when Spring Boot becomes linked to the relational database containing the pull request information since Spring Boot makes it incredibly simple to make queries or to mutate the data inside the database tables, if necessary.

Chapter 1: Introduction

Visualising complex and congested data is a challenge, no matter how well-versed a developer might be in APIs. Therefore, to help aid in this process, several APIs and frameworks can be used to help make the process easier and more efficient. For instance, Spring Boot is used for the backend of the system. This is because it Spring Boot makes programming REST APIs incredibly simple with its several layers of abstraction it implements. In addition to this, it is powered by Java which is a programming language that is universally used and appreciated by developers, and thus making the project more accessible to contributors. On the other hand, D3.js, React, Material UI are used for the frontend, in addition to GraphQL. These APIs and frameworks are chosen for their simplicity and professional look. GraphQL supplements the crawling process to collect the necessary data from GitHub. Without this, the relational database would contain no data and migration would not be possible. React and Material UI go hand-in-hand for polished and high-quality websites. Moreover, these libraries give developers an immense array of tools for lots of different styles of project. Finally, D3.js is arguably the most important part of the project specification since it is the library that encompasses the visualisation aspect of the project. D3 is incredibly efficient to use, and yet so powerful in nature. It can support thousands of nodes whilst seamlessly reading database from the backend in a matter of seconds.

To summarise, the following APIs, frameworks, and libraries are chosen to support the design and development of the project:

1. GraphQL
2. React
3. Material UI
4. D3.js
5. Spring Boot

1.1 Project Prototype Screenshot

Most final reports will contain a mixture of figures and charts along with the main body of text. The figure caption should appear directly after the figure as seen in Figure 1.1 whereas a table caption should appear directly above the table. Figures, charts, and tables should always be centered horizontally.



Figure 1.1: Logo of RHUL.

Chapter 2: Crawler

Crawling the necessary information was the very first task for the project. To do this, the GitHub GraphQL API was used. The code for this milestone was split up into three Python files. The first one was the main `crawl.py` file containing the code for making the API calls to the GitHub GraphQL API endpoint. The second one was the `logger.py` file to initialise the logger, so appropriate messages could be outputted to the console. The second and final one was the `split.py` file that split each individual crawled pull request into its own JSON file. The `logger.py` and `split.py` files were relatively straight forward to code and test, since they both performed simple functions. However, the `crawl.py` was a lot more complex in nature and, thus, was more problematic to write. This was the first problem encountered during this milestone.

In fact, there are two versions of this `crawl.py` file that have to be written during development. The first version is one that does not store the "cursor" value of the last pull request that it fetches, so it is not as efficient. A "cursor" value is effectively the unique identifier of each pull request as it is being fetched - it is NOT the ID of the pull request itself. The common issue with this first version of the crawler is that it uses up the 5,000 API calls per hour that a user gets using the GitHub GraphQL API. It also does this without retrieving the necessary amount of data (in this case, over 10,000 pull requests). As a result of this, the second version does store the "cursor" value of the last pull request that it fetches, and thus is a lot more efficient with its API calls. Only 1,000–2,000 API calls are required when using this second, improved version of the crawler, compared to all 5,000 API calls when using the original version.

The second problem encountered during this milestone was formatting and structuring the query string that would be sent as part of the callback to the GitHub GraphQL API endpoint. The format of this query string has to be very precise for the endpoint to accept it and not report back errors during compilation. After adjusting the formatting of this query string, the crawler does eventually function as expected.

Overall, the final, improved crawler file that is produced uses the `requests` pip library to make a customised callback to the GitHub GraphQL API endpoint, using its own built-in methods, and uses the response from this callback to generate a collection of JSON files containing 10 pull requests per file. The crawler can accommodate for a single repository as input, as well as a dictionary of key-value pairs linking the owner of a repository with the repository's name. As this `crawl` file is running, messages are outputted to the user in the console using the logger that was initialised earlier on. These files can then be read by the `split.py` file to make a JSON file for each pull request, ready for the next milestone of the project: Database.

Chapter 3: **Database**

Test

Chapter 4: **Frontend**

Test

Chapter 5: **Backend**

Test

Chapter 6: **Conclusion**

Test